

✓ LAB 6:

ATTENTION-MECHANISM TRANSFORMER ARCHITECTURE

```
# Setup dependencies in CoLab
!pip install torch==2.3.1 torchvision torchaudio
!pip install -U torchtext==0.18.0
!python -m spacy download en_core_web_sm
!python -m spacy download fr_core_news_sm
```

[Show hidden output](#)

```
# Get necessary text files from repo
!wget https://raw.githubusercontent.com/j4lando/phys417/main/Lab6/en_to
!wget https://raw.githubusercontent.com/j4lando/phys417/main/Lab6/de_to
```

[Show hidden output](#)

```
%matplotlib inline

import numpy as np
import math
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from typing import Iterable, List
from timeit import default_timer as timer

import torch
import torch.nn as nn
from torch.nn import Transformer
from torch import Tensor

from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator
from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence

np.random.seed(0)
torch.manual_seed(0)

DEVICE = torch.device('mps' if torch.backends.mps.is_available() else
print(DEVICE)
```

```
cuda
/usr/local/lib/python3.12/dist-packages/torchtext/data/__init__.py:4: I
/!\ IMPORTANT WARNING ABOUT TORCHTEXT STATUS /\
Torchtext is deprecated and the last released version will be 0.18 (th
warnings.warn(torchtext._TORCHTEXT_DEPRECATION_MSG)
/usr/local/lib/python3.12/dist-packages/torchtext/vocab/__init__.py:4:
/!\ IMPORTANT WARNING ABOUT TORCHTEXT STATUS /\
Torchtext is deprecated and the last released version will be 0.18 (th
warnings.warn(torchtext._TORCHTEXT_DEPRECATION_MSG)
/usr/local/lib/python3.12/dist-packages/torchtext/utils.py:4: UserWarn
/!\ IMPORTANT WARNING ABOUT TORCHTEXT STATUS /\
Torchtext is deprecated and the last released version will be 0.18 (th
warnings.warn(torchtext._TORCHTEXT_DEPRECATION_MSG)
```

For more on PyTorch Transformer support for language translation:

https://pytorch.org/tutorials/beginner/translation_transformer.html

✓ Download the tokenization library

We will learn what these are later. For now, your goal is to run the following code.

```
en_token_transform = get_tokenizer('spacy', language='en_core_web_sm')
fr_token_transform = get_tokenizer('spacy', language='fr_core_news_sm')
```

If you get an error saying something along the lines of "the computer can't find this", it's because you haven't downloaded the tokenization libraries yet. To do that, you should open up command terminal and run the following code:

```
python -m spacy download en_core_web_sm
```

```
python -m spacy download fr_core_news_sm
```

Bada bing bada boom

✓ Download data

```
# Load data as a list of tuples
with open("en_to_fr.txt", "r", encoding="utf-8") as f:
    lines = f.read().split("\n")

text_pairs = []
for l in lines:
    appendage = l.split("\t")
    text_pairs.append(appendage)

# Print the first ten samples
for i in range(10):
    print(text_pairs[i][0], text_pairs[i][1])
```

```
Hi. Salut!
Stop! Arrête-toi !
I won! J'ai gagné !
Get up. Lève-toi.
Hop in. Montez.
I paid. J'ai payé.
No way! Il n'en est pas question !
We won. Nous gagnâmes.
Be fair. Soyez juste !
Be nice. Soyez gentils !
```

✓ Create source and target language tokenizers

```
# Download source and target language tokenizers
SRC_LANGUAGE = 'en'
TGT_LANGUAGE = 'fr'

token_transform = {}
token_transform[SRC_LANGUAGE] = en_token_transform
token_transform[TGT_LANGUAGE] = fr_token_transform
# hint: if you get an OSError, try running the following lines from tl
# python -m spacy download en_core_web_sm
# python -m spacy download fr_core_news_sm

en_0_tokenized = token_transform[SRC_LANGUAGE](text_pairs[0][1])
fr_0_tokenized = token_transform[TGT_LANGUAGE](text_pairs[0][0])

# Print the tokenized first line of each dataset
print(en_0_tokenized)
print(fr_0_tokenized)

['Salut', '!']
['Hi', '.']
```

- ✓ Create a vocabulary for each language's dataset

```

# Special tokens: a "unique" filler-token to use when we can't tokeni:
special_tokens = ['<unk>', '<pad>', '<bos>', '<eos>']

# And their special indeces in our vocabulary
UNK_IDX, PAD_IDX, BOS_IDX, EOS_IDX = 0, 1, 2, 3

# A helper function that converts a list of strings into a list of li:
def yield_tokens(data_iter, language):
    language_index = {SRC_LANGUAGE: 0, TGT_LANGUAGE: 1}

    for data_sample in data_iter:
        try:
            yield token_transform[language](data_sample[language_index[
        except IndexError:
            print(f"token_transform.keys(): {token_transform.keys()}")
            print(f"language: {language}")
            print(f"data_sample: {data_sample}")
            print(f"language_index: {language_index}")
            raise IndexError

# Create a vocabulary object for each language using the build_vocab_
vocab_transform = {}
for ln in [SRC_LANGUAGE, TGT_LANGUAGE]:
    # Invoke torchtext's Vocab object
    vocab_transform[ln] = build_vocab_from_iterator(yield_tokens(text
                                                    min_freq=1,
                                                    specials=special_
                                                    special_first=True

# ``UNK_IDX`` is the index returned when the token is not found.
# If we don't use this, we'll get a ``RuntimeError`` when the queried
for ln in [SRC_LANGUAGE, TGT_LANGUAGE]:
    vocab_transform[ln].set_default_index(UNK_IDX)

# Let's see the first 20 words in each vocabulary
print(vocab_transform[SRC_LANGUAGE].get_itos()[:20])
print(vocab_transform[TGT_LANGUAGE].get_itos()[:20])

['<unk>', '<pad>', '<bos>', '<eos>', '.', 'I', 'you', 'to', '?', 'the'
['<unk>', '<pad>', '<bos>', '<eos>', '.', 'de', 'Je', '?', 'pas', 'est

```

✓ Train-Validate-Test split

```
# Shuffle the text pairs
shuffler = np.random.permutation(len(text_pairs))
text_pairs = [text_pairs[i] for i in shuffler]

# Train-validate-test split: 80-10-10
n_train = int(0.8*len(text_pairs))
train_pairs = text_pairs[:n_train]

n_val = int(0.1*len(text_pairs))
val_pairs = text_pairs[n_train:n_train+n_val]

n_test = int(0.1*len(text_pairs))
test_pairs = text_pairs[n_train+n_val:]

print(f"{len(text_pairs)} total pairs")
print(f"{len(train_pairs)} training pairs")
print(f"{len(val_pairs)} validation pairs")
print(f"{len(test_pairs)} test pairs")
```

```
17563 total pairs
14050 training pairs
1756 validation pairs
1757 test pairs
```

✓ Mask functions

```
# Helper
def generate_square_subsequent_mask(sz):
    mask = (torch.triu(torch.ones((sz, sz), device=DEVICE)) == 1).trai
    mask = mask.float().masked_fill(mask == 0, float('-inf')).masked_
    return mask

# Mask function
def create_mask(src, tgt):
    src_seq_len = src.shape[0]
    tgt_seq_len = tgt.shape[0]

    tgt_mask = generate_square_subsequent_mask(tgt_seq_len)
    src_mask = torch.zeros((src_seq_len, src_seq_len), device=DEVICE).

    src_padding_mask = (src == PAD_IDX).transpose(0, 1)
    tgt_padding_mask = (tgt == PAD_IDX).transpose(0, 1)
    return src_mask, tgt_mask, src_padding_mask, tgt_padding_mask
```

✓ Collation

```

# A helper function to club together sequential operations
def sequential_transforms(*transforms):
    def func(txt_input):
        for transform in transforms:
            txt_input = transform(txt_input)
        return txt_input
    return func

# Adds BOS/EOS and creates a tensor for input sequence indices
def tensor_transform(token_ids):
    return torch.cat((torch.tensor([BOS_IDX]),
                        torch.tensor(token_ids),
                        torch.tensor([EOS_IDX])))

# ``src`` and ``tgt`` language text transforms convert the raw strings
text_transform = {}
for ln in [SRC_LANGUAGE, TGT_LANGUAGE]:
    text_transform[ln] = sequential_transforms(token_transform[ln], #
                                                vocab_transform[ln], #
                                                tensor_transform) # Add

# The "collation" function collates data samples into batch tensors
def collate_fn(batch):
    src_batch, tgt_batch = [], []
    for src_sample, tgt_sample in batch:
        src_batch.append(text_transform[SRC_LANGUAGE](src_sample.rstr:
        tgt_batch.append(text_transform[TGT_LANGUAGE](tgt_sample.rstr:

    src_batch = pad_sequence(src_batch, padding_value=PAD_IDX)
    tgt_batch = pad_sequence(tgt_batch, padding_value=PAD_IDX)
    return src_batch, tgt_batch

```

✓ Training, evaluation functions

```

# This function trains the model for a single epoch
def train_epoch(model, optimizer):
    model.train()
    loss_list = []
    train_iter = train_pairs
    train_dataloader = DataLoader(train_iter, batch_size=BATCH_SIZE,

```

```

for src, tgt in train_dataloader:
    src = src.to(DEVICE)
    tgt = tgt.to(DEVICE)

    tgt_input = tgt[:-1, :]

    # Since we're training, recall we need to mask our input
    src_mask, tgt_mask, src_padding_mask, tgt_padding_mask = create_masks(src, tgt_input)

    # What do you think the model does with the masks when it's in training mode?
    logits = model(src, tgt_input, src_mask, tgt_mask, src_padding_mask, tgt_padding_mask)

    optimizer.zero_grad()

    tgt_out = tgt[1:, :]
    loss = loss_fn(logits.reshape(-1, logits.shape[-1]), tgt_out.reshape(-1))
    loss.backward()

    optimizer.step()
    loss_list.append(loss.item())

return loss_list

# Evaluates the model
def evaluate(model):
    model.eval()
    loss_list = []

    val_iter = val_pairs
    val_dataloader = DataLoader(val_iter, batch_size=BATCH_SIZE, collate_fn=collate_fn)

    for src, tgt in val_dataloader:
        src = src.to(DEVICE)
        tgt = tgt.to(DEVICE)

        tgt_input = tgt[:-1, :]

        src_mask, tgt_mask, src_padding_mask, tgt_padding_mask = create_masks(src, tgt_input)

        logits = model(src, tgt_input, src_mask, tgt_mask, src_padding_mask, tgt_padding_mask)

        tgt_out = tgt[1:, :]
        loss = loss_fn(logits.reshape(-1, logits.shape[-1]), tgt_out.reshape(-1))
        loss_list.append(loss.item())

```

```
return loss_list
```

✓ Define model

```
# The PositionalEncoding module quantifies the relative position of words
# Notice that this is not actually an MLP or neural network, i.e. it is
# it is just a function that you could represent analytically, if you like
class PositionalEncoding(nn.Module): # <-- "Embedding"
    def __init__(self, emb_size, dropout, maxlen = 5000):
        super(PositionalEncoding, self).__init__()
        den = torch.exp(- torch.arange(0, emb_size, 2)* math.log(10000/(emb_size)))
        pos = torch.arange(0, maxlen).reshape(maxlen, 1)
        pos_embedding = torch.zeros((maxlen, emb_size))
        pos_embedding[:, 0::2] = torch.sin(pos * den)
        pos_embedding[:, 1::2] = torch.cos(pos * den)
        pos_embedding = pos_embedding.unsqueeze(-2)

        self.dropout = nn.Dropout(dropout)
        self.register_buffer('pos_embedding', pos_embedding)

    def forward(self, token_embedding: Tensor):
        return self.dropout(token_embedding + self.pos_embedding[:token_embedding.size(0)])

# The TokenEmbedding module converts a tensor of vocabulary-indices into
# Also not a neural network, but a lookup table
class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size, emb_size):
        super(TokenEmbedding, self).__init__()
        self.embedding = nn.Embedding(vocab_size, emb_size)
        self.emb_size = emb_size

    def forward(self, tokens: Tensor):
        return self.embedding(tokens.long()) * math.sqrt(self.emb_size)

# Sequence-to-sequence transformer
class Seq2SeqTransformer(nn.Module):
    def __init__(self,
                  num_encoder_layers,
                  num_decoder_layers,
                  embedding_size,
                  num_heads,
```

```

        src_vocab_size,
        tgt_vocab_size,
        dim_feedforward = 512,
        dropout = 0.1):
    super(Seq2SeqTransformer, self).__init__()
    self.transformer = Transformer(d_model=embedding_size,
                                   nhead=num_heads,
                                   num_encoder_layers=num_encoder_
                                   num_decoder_layers=num_decoder_
                                   dim_feedforward=dim_feedforward,
                                   dropout=dropout)

    self.generator = nn.Linear(embedding_size, tgt_vocab_size)
    self.src_tok_emb = TokenEmbedding(src_vocab_size, embedding_size)
    self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, embedding_size)
    self.positional_encoding = PositionalEncoding(
        embedding_size, dropout=dropout)

def forward(self,
            src,
            trg,
            src_mask,
            tgt_mask,
            src_padding_mask,
            tgt_padding_mask,
            memory_key_padding_mask):
    src_emb = self.positional_encoding(self.src_tok_emb(src)) # src
    tgt_emb = self.positional_encoding(self.tgt_tok_emb(trg)) # trg
    outs = self.transformer(src_emb, tgt_emb, src_mask, tgt_mask,
                            src_padding_mask, tgt_padding_mask, memory_key_padding_mask)
    return self.generator(outs)

def encode(self, src: Tensor, src_mask: Tensor):
    return self.transformer.encoder(self.positional_encoding(
        self.src_tok_emb(src)), src_mask)

def decode(self, tgt: Tensor, memory: Tensor, tgt_mask: Tensor):
    return self.transformer.decoder(self.positional_encoding(
        self.tgt_tok_emb(tgt)), memory,
        tgt_mask)

```

✓ Define hyperparameters

```

SRC_VOCAB_SIZE = len(vocab_transform[SRC_LANGUAGE])
TGT_VOCAB_SIZE = len(vocab_transform[TGT_LANGUAGE])
EMB_SIZE = 512
NUM_HEADS = 8 # Why 8? What do you expect to happen if we increase th
FFN_HID_DIM = 512
BATCH_SIZE = 32
NUM_ENCODER_LAYERS = 3
NUM_DECODER_LAYERS = 3
NUM_EPOCHS = 6

```

✓ Instantiate model, loss function, optimizer

```

transformer = Seq2SeqTransformer(NUM_ENCODER_LAYERS, NUM_DECODER_LAYERS,
                                NUM_HEADS, SRC_VOCAB_SIZE, TGT_VOCAB_SIZE)

for p in transformer.parameters():
    if p.dim() > 1:
        nn.init.xavier_uniform_(p)

transformer = transformer.to(DEVICE)

loss_fn = torch.nn.CrossEntropyLoss(ignore_index=PAD_IDX)

optimizer = torch.optim.Adam(transformer.parameters(), lr=0.0001, beta=(0.9, 0.999))

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/transformer.py:100:
  warnings.warn(f"enable_nested_tensor is True, but self.use_nested_tensor is False")

```

✓ Train the model

```
# Fair warning: you might get an "out of memory" error
# If that happens, try reducing the batch size

train_loss_list = []
val_loss_list = []
for epoch in range(1, NUM_EPOCHS+1):
    start_time = timer()
    train_loss = train_epoch(transformer, optimizer)
    train_loss_list.extend(train_loss)
    end_time = timer()
    val_loss = evaluate(transformer)
    val_loss_list.extend(val_loss)
    print((f"Epoch: {epoch}, Epoch time = {(end_time - start_time):.3s}"))
```

```
/usr/local/lib/python3.12/dist-packages/torch/nn/functional.py:5137: UserWarning: Using a data path and weights that has been converted from torchscript.
warnings.warn(
Epoch: 1, Epoch time = 16.334s
Epoch: 2, Epoch time = 16.111s
Epoch: 3, Epoch time = 16.371s
Epoch: 4, Epoch time = 16.662s
Epoch: 5, Epoch time = 17.045s
Epoch: 6, Epoch time = 17.495s
```

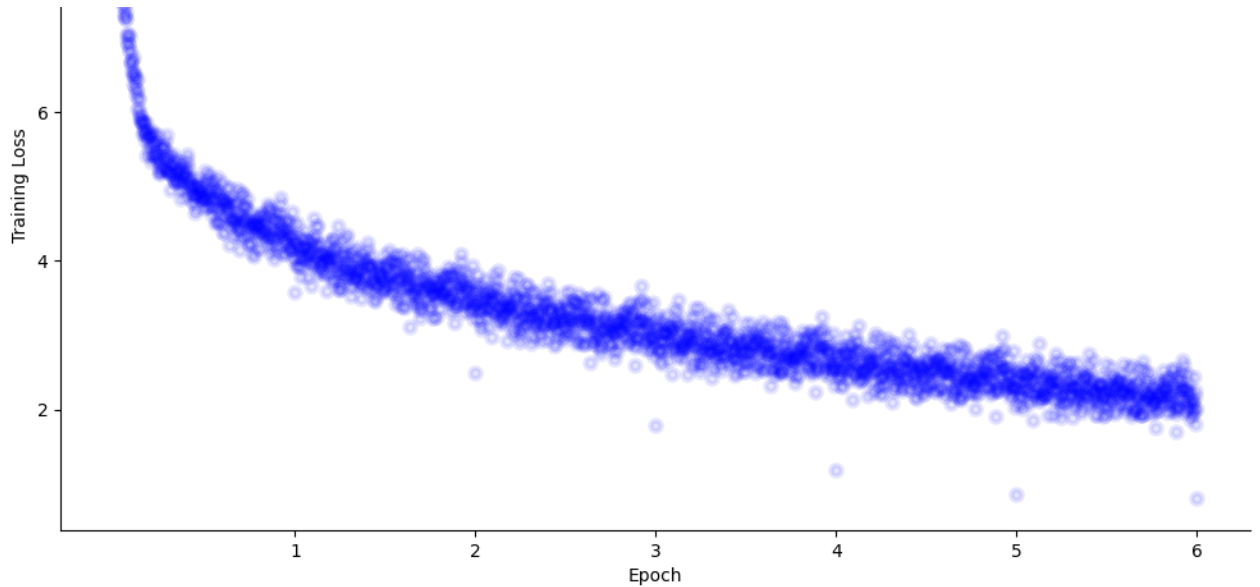
✓ Visualize and Evaluate the Model

```
# Plot the loss
plt.figure(figsize = (12, 7))

plt.scatter(range(len(train_loss_list)), train_loss_list, color = 'blue')
plt.ylabel("Training Loss")
plt.xlabel("Epoch")
plt.xticks(ticks = [(i+1)*len(train_loss_list)//NUM_EPOCHS for i in range(NUM_EPOCHS)])
plt.legend()
sns.despine()
```

```
/tmp/ipykernel_22388/3952521802.py:8: UserWarning: No artists with labels found in the legend (legend entries are empty).
plt.legend()
```





Print the last 10 target/translated sequences from test set

```
# Function to generate output sequence using greedy algorithm
# This basically saves us some compute time by taking a bunch of short
def greedy_decode(model, src, src_mask, max_len, start_symbol):
    src = src.to(DEVICE)
    src_mask = src_mask.to(DEVICE)

    memory = model.encode(src, src_mask)
    ys = torch.ones(1, 1).fill_(start_symbol).type(torch.long).to(DEVICE)
    for i in range(max_len-1):
        memory = memory.to(DEVICE)
        tgt_mask = (generate_square_subsequent_mask(ys.size(0)))
                    .type(torch.bool)).to(DEVICE)
```

```

        out = model.decode(ys, memory, tgt_mask)
        out = out.transpose(0, 1)
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.item()

        ys = torch.cat([ys,
                        torch.ones(1, 1).type_as(src.data).fill_(next_
        if next_word == EOS_IDX:
            break
    return ys

# Actual function to translate input sentence into target language
# Translation function that actually uses the model to translate a sei
def translate(model, src_sentence):
    model.eval()
    src = text_transform[SRC_LANGUAGE](src_sentence).view(-1, 1)
    num_tokens = src.shape[0]
    src_mask = (torch.zeros(num_tokens, num_tokens)).type(torch.bool)
    tgt_tokens = greedy_decode(
        model, src, src_mask, max_len=num_tokens + 5, start_symbol=B
    return " ".join(vocab_transform[TGT_LANGUAGE].lookup_tokens(list(

for i in range(10):
    test_pair = test_pairs[-i]
    test_str_de = test_pair[0]
    test_str_en = test_pair[1]
    print(f"Target: {test_str_en}")
    print("Model output:", translate(transformer, test_str_de))
    print("")

```

Target: Le vieux couple était assis côte-à-côte.
 Model output: Les gens ont pris ses parents en retard .

Target: Vous devez vous lever.
 Model output: Tu dois me rendre .

Target: Il va à Karuizawa chaque année.
 Model output: Il se trouve le travail à l' année .

Target: Vois ce que tu peux concocter.
 Model output: À ce que tu peux venir avec toi .

Target: Elles veulent savoir quel est ton plan.
 Model output: Ils veux savoir ce qu' est ton plan .

Target: C'est la partie que j'ai préférée.

Model output: C' est la fenêtre de mieux de mieux .

Target: Je pris un taxi car le car était en retard.

Model output: J' ai pris un taxi dans le bus en retard .

Target: Il a finalement accompli ce qu'il avait entrepris.

Model output: Il a pris un peu de ce qu' il a fait .

Target: J'aimerais m'entretenir avec vous de ce qui a eu lieu.

Model output: J' aimerais te parler de ce qui est arrivé .

Target: Les étudiants étaient mal à l'aise avant l'examen.

Model output: Les étudiants ont été presque presque en premier avant